



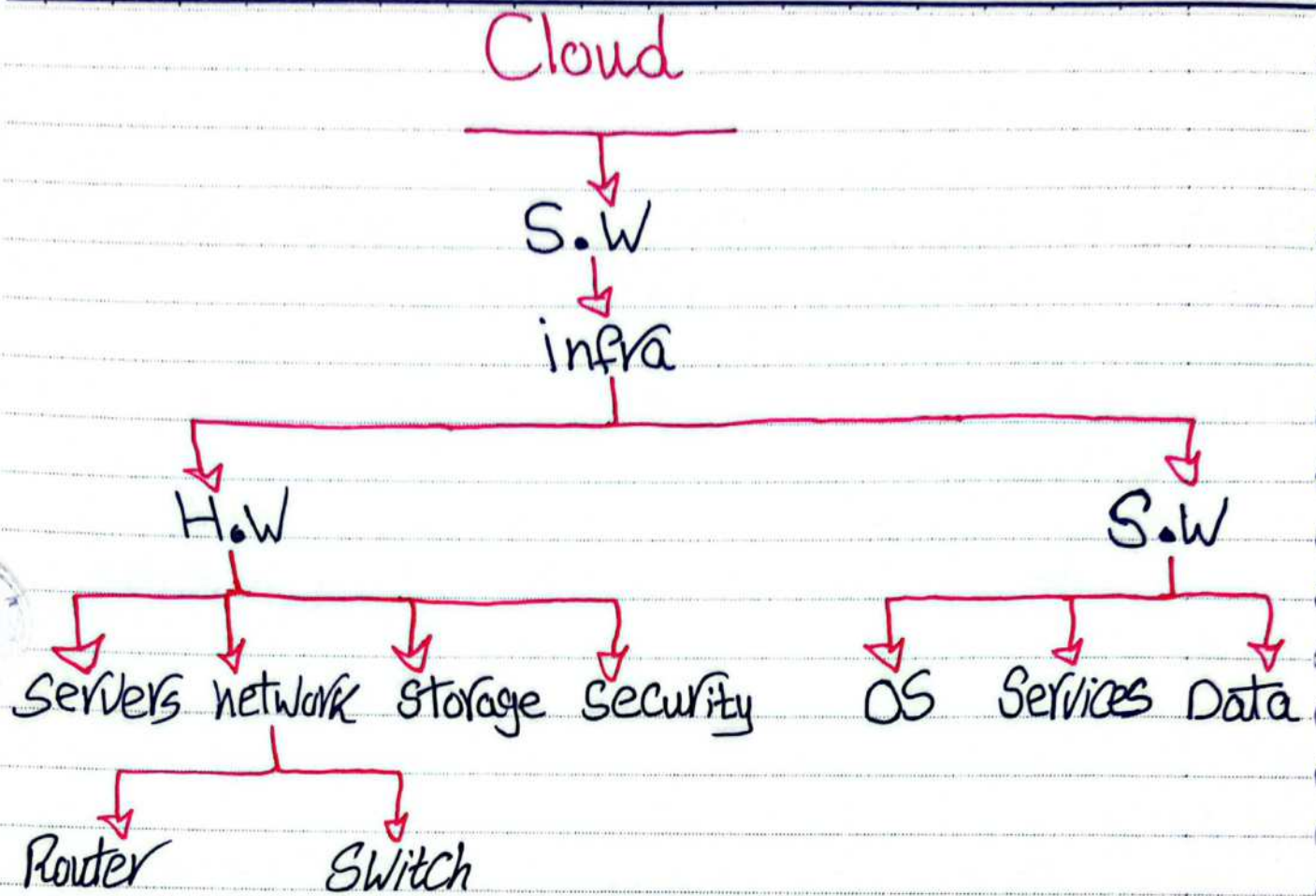
Cloud

- # لازم انك كـ DevOps تكون عارف Cloud
- انك كـ DevOps مسؤول عن الـ infrastructure
- الـ onPrem يعني الـ server الـ Physical
- الـ Cloud

- # ميفيش نقول انك كـ DevOps تعرف الـ onPrem
- ومتعرفش الـ Cloud

- # شرط اساسي تكون عارف Cloud
- عشان كذا من غير للاسف مش هتبقى Profile DevOps

- # أي Technology بنستخدمها الهدف منها أي ؟
- نوفر بيئة مناسبة للـ S.W عشان يشتغل أو Operating
- أي Team يشتغل لخدمة الـ S.W ويكون Accessible/Available



المنظومة التي بتدير ال **Infrastructure** هي ال **Data Center**
 # ال **Data Center** المكان التي ب **Host** كل ال **Infrastructure**

ممكن ال **Data Center** تكون جهاز صغير في مكان صغير
 - ممكن يكون مبنى وممكن يكون عدة صاباني
 - اول لما تبقي عدة صاباني ساعتها بيبي اسمها **Zone**

اقل **Scale** عشان اقدر اعمل **Data Center**
 - 6 شهور على الأقل

- سنة لو ال **Scale** متوسطه

- سنتين ل 5 سنين لو ال **Scale** كبيره

أي **Scale** بتفلمس بتحتاج تسكندل تدريبي اقل حاجة **سنة**

اقدر تكلفه لل **Data Center** به ملايين حتى لو **mini Center**



Cloud

اكيد ال **Data Center** مش هتشتغل لو صدها دانا هكون محتاج هو ظنين متفهمين في جميع المجالات

نسبة نجاح المشروع بتاعك اكيد هتكون 50/50 %
- لو المشروع فشل

- عندي القدرة اتعمل الخسائر وارجع من ثاني
- مهندس القدرة على تحمل الخسائر ساعتها كل حاجة **الله**
- لو المشروع نجح
- هحتاج اطور واكبر

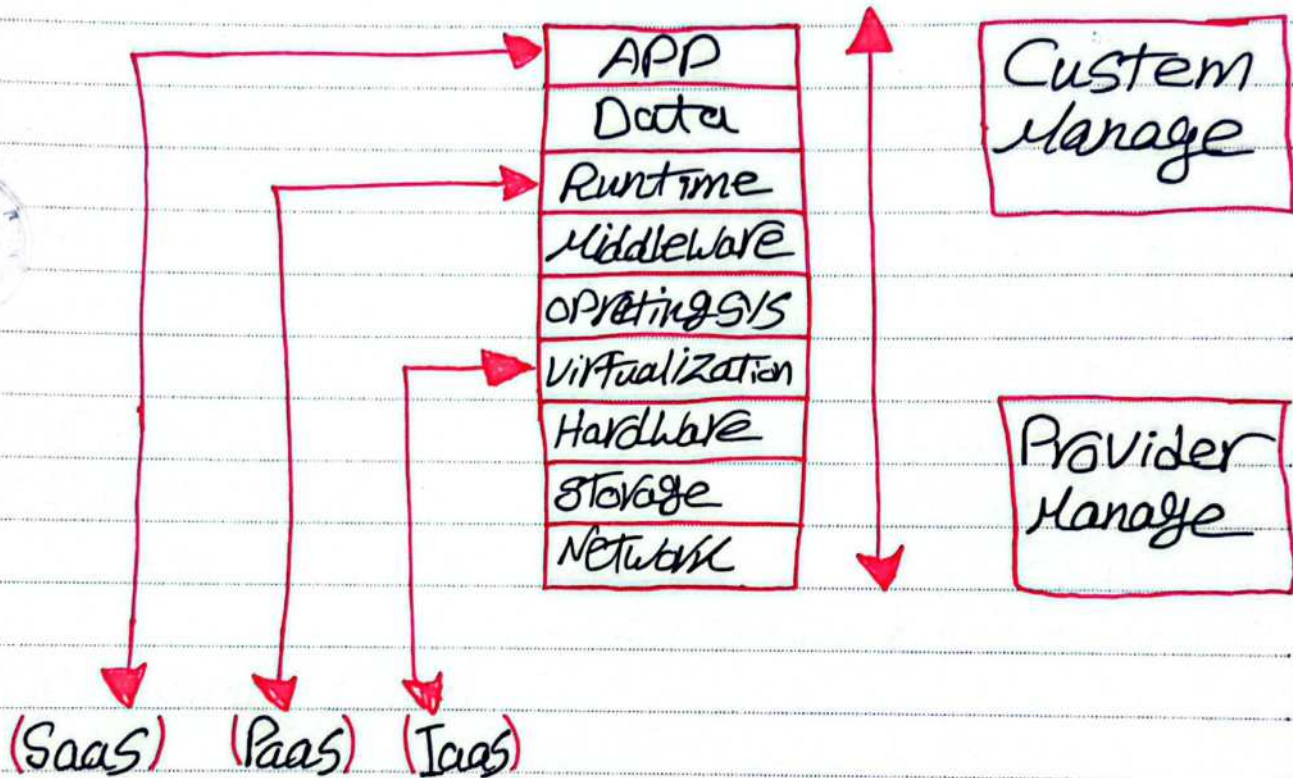
عشان نعمل **Infrastructure** أو **Data Centers**
- هنروح لأصحاب الأموال وهيدعمونا بالفلوس فقط
- لازم نختار مكان كويس ومناسب / ليه دراسه وليها متفهمين
- لازم يكون عندي أكثر من مصدر طاقة للكهرباء
- طاقة شمسية - طاقة رياح - كهرباء عمومية
- مكان بعيد عن الكوارث الطبيعية أو الصناعية
- انا اول انعمل ال **Data Center** قريبة من العملاء داهيوفر فلوس واهي **High Availability**

بعد تجهيز ال **Data centers**
- هنستفيد من ال **Cloud** أو ال **Technology**
- ازاي اوصل ~~من~~ من فلان ال **Internet**
- ازاي هستخدمها من فلان **Virtualization**

- اكيد انت مش هتستخدم ال **Data Center** كلها انت
- هتحتاج جروب بسيط منها وتنا ربي دور ال **Virtualization**
- بالتالي مش هرفع غير اللي هتستخدمه فقط مثلاً في الساعة 16
- لو خسرت هفقد الآشياء عاري وفجئاً انا مش بخسر
- كل اللي خسرتة هي قدر استهلاكك من ال **Resources** اللي حيتوزها

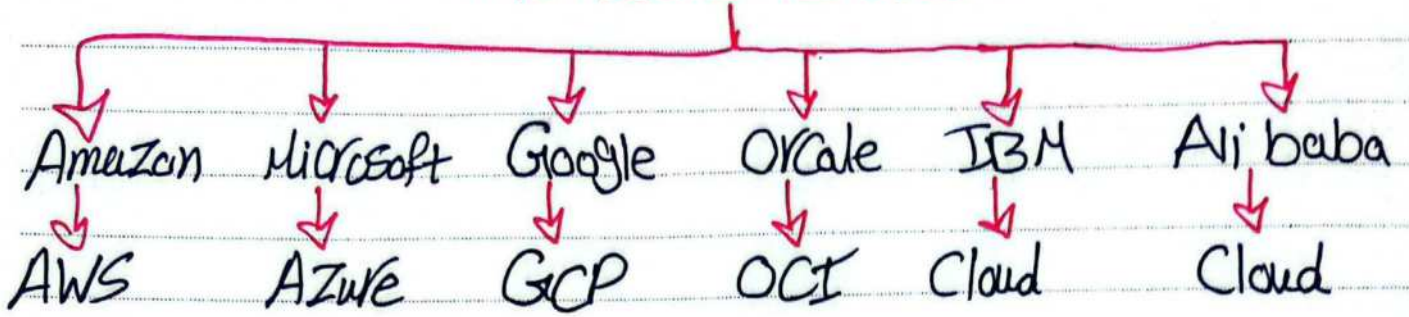
Cloud

بعد التطوير كله بدأ يظهر لنا **Models**
 أي **System** أو **Server** سيكون من



- Infrastructure as a Service (IaaS)** ← الـ **User** يياض **Resources** فقط وانا بعمل الـ **System** بناغي
- Platform as a Service (PaaS)** ← الـ **User** يياض **System Running** وابدأ انزل فيه الـ **Project** بناغي
- Software as a Service (SaaS)** ← الـ **User** صيعلش حاجة هو يياض **Service** جاهزة ويدفع مقابل الاستخدام مثل **Zoom meeting**

Providers Cloud



كل Provider يقدم Services وكل واحد فيهم مميز وفي service معينة سواء من ناحية ال Cost أو غيره

Types Cloud

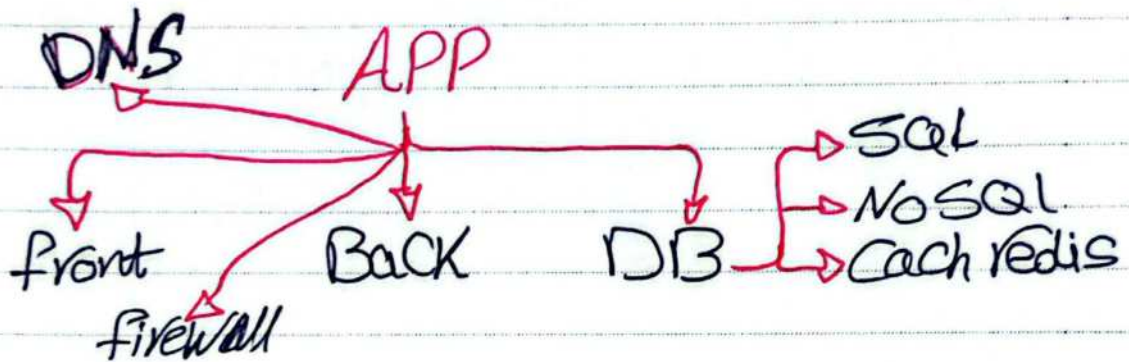
- 1 Public
 - 2 Private Cloud → on-Prim
 - 3 HyPrid.
 - 4 Multi Cloud
 - 5 Community Cloud
- جزء on-Prim وجزء Cloud
 يستخدم أكثر من Provider
 - لو اتوافق أكثر من Provider فممكن لو أي Provider وقع الخدمات عندي متوقعة كلها

Private + Public

كلمة السريين ال
 - مين اللي بيملك لو انت اللي بيملك وتدير بين Private
 لو انت اللي بتدير فقط والملكية مش بتاعتك بين Public

Cloud

لو عايز افوت عندي Software



لو رفس لا Provider هلاقية بيقدمني ال Services جاهزة
- يعني عنده DB فيها ال 3 انواع جاهزين

لازم اكون عارف ال architecture بتاعني أي عشان افتر الخدمات

AWS Architecture

(AZ) Availability zone ①

Region City ② ← مجموعة من ال Zone في

مدينة واحدة وببساطة عدد ال Zone في ال Region من [3+6]

Edge Location ③

مسؤولة عن ال Content + Caching + Media

ال Media بتحتاج Traffic عالي جداً هيفوتس لنا

يبقى في حد في مصر أمولة على edge loc في أمريكا

الفكرة كلها اني هوله لأقرب edge location

Services AWS

في خدماته لا AWS

- عشان أقوم APP محتاج VM + DISK لو أنا On-Prem
- أمازون وفرت service بملها تقوى VM بنفسه

على طوره من فلك ال Cloud
في service اسمها EC2 بتوفرلك DISK + VMS

- لو محتاج DB لو SQL في service DynamoDB

- لو محتاج DB لو NOSQL في service RDS

- لو محتاج Network service VPC

- لو عايز DB Cash Elastic Cache

- لو عايز Load Balancing ELB

- لو عايز DNS Route53

- عايز مكان أفتر فيه الداتا الأساسية S3 service

- أي هو نوع الملفات اللي فالداتا ؟

- الملفات ال user اللي بيرفعها سواء صورة أو فيديو

- لو ال user رفع صورة أو فيديو هند سياسة الاستخدام

سواء كانت محتوى علف ملكة Recognition Service

- لما user يدخل ممكن يكون معاه Device مش بيدعم كل حاجة

يعني موبيل مش بيدعم ال 2K أو ال HD ساعة

هتحتاج service اسمها Lambda بتعمل جوة الصورة

لتتناسب مع ال Device اللي معاه

- لو معاك بقى جهاز بيدعم 2K ساعة هتوصل

Database كاتبيه

- لو انت في أمريكا هتوديك ال Edge location بتاعة أمريكا

وال service اللي هتوديك ال Edge location Cloud Front

- لما user يبحث عن حاجة معينة أو يفتح اعلان لحاجة معينة

ساعتها ال Runtime كله عن الحاجة اللي يبحث عنها

ال service Kinesis بتخزن لوصفها في مكان

Services AWS

- لما افزت ال **Data** الخاصة بـ **Kinesis Service** بس هتخزن فين؟ هنا يجي دور ال **Big Data** ينظمها ويرتبها
- ينظمها مثلاً كل اعلانات التريبال في **Section** لوصف
- مش اشف الاعلانات اقرب اعلان للتريبال ال **Client**
- مع كل ال **Data** الفخمة محتاجين **Tool** للإدارة والتنظيم يعني دور **SPARK/HADOOP Tools**
- بالنسبة لـ **Big Data** ليها **Tools** ← **EMR, Glue**

AWS vs Azure vs GCP

EC2

Virtual Machine

Compute Engine

RDS

SQL DB

Cloud Database

Kinesis

Stream analysis

Cloud Dataflow

Cloud & AWS — Visual Notes

مرجع مرئي متكامل مبني على مذكرة الـ **Cloud** بخط اليد — كل موضوع في صفحة: شرح مختصر، **diagram** واضحة، وشرح مفصل تحتها، وفي الآخر **diagram** شاملة يترابط كل حاجة.

14 topics

15 diagrams

On-Prem • Cloud

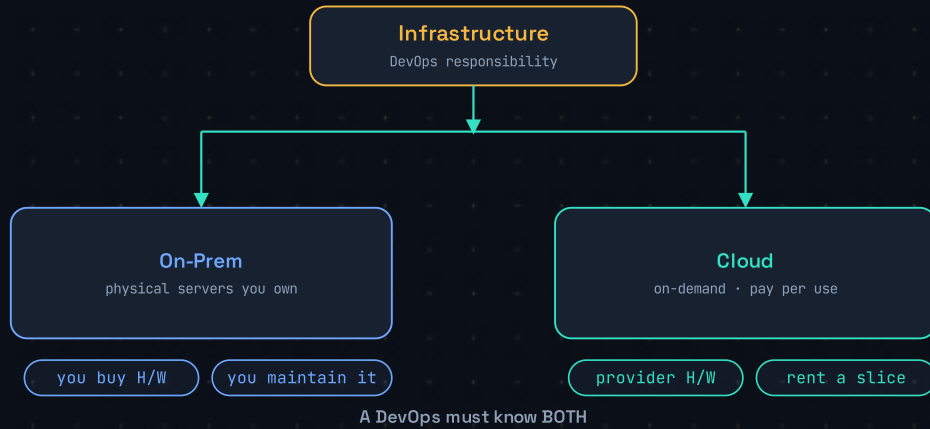
IaaS • PaaS • SaaS

AWS Services

01 On-Prem vs Cloud

Cloud • Basics

الـ **DevOps** مسؤول عن الـ **Infrastructure**، وعنده طريقتين يشغلها بيهم: **On-Prem** أو **Cloud** — ولازم يعرف الاثنين.



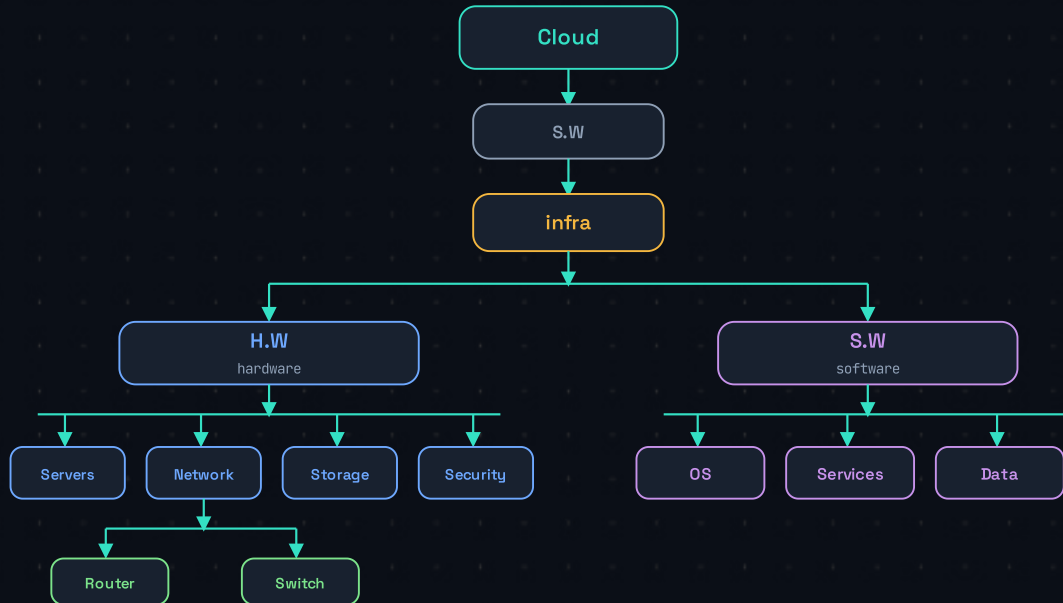
KEYNOTE • شرح مفصل

شغلة الـ **DevOps** الأساسية إنه المسؤول عن الـ **Infrastructure** (البنية التحتية) اللي بيشتغل عليها الـ **Software**. وعشان توصل الـ **S.W** ده ويبقى **Available** و **Accessible** لأي **team**، قدامك طريقتين:

- **On-Prem**: السيرفرات الفيزيائية (**Physical servers**) موجودة عندك انت — انت اللي اشتريتها وانت المسؤول عن صيانتها وتشغيلها.
- **Cloud**: البنية التحتية عند مزود الخدمة، وانت بتأجر جزء منها وتدفع على قد استهلاكك بس.

شرط أساسي: مينفعش تقول انك **DevOps** وانت متعرفش الـ **On-Prem** ولا الـ **Cloud** — ده جزء أساسي في الـ **profile** بتاع أي **DevOps**.

أي بيئة Cloud يتبني من S.W و infra، والـ infra نفسها بتقسم H.W و S.W — دي الخريطة اللي لازم تكون في دماغك.



KEYNOTE • شرح مفصل

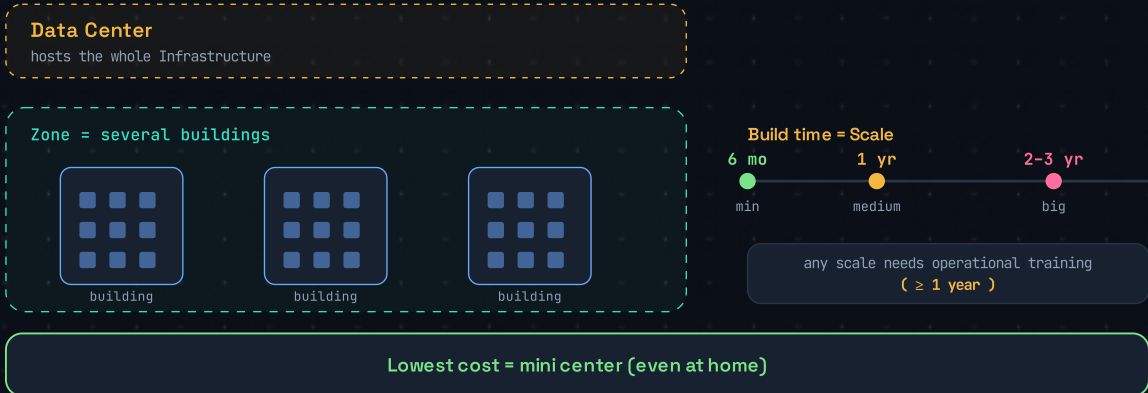
دي شجرة البنية التحتية بالكامل من فوق لتحت، الـ Cloud بيتبني من طبقتين كبار: S.W والـ infra، والـ infra بتتفرّع لـ Hardware و Software:

• H.W فيه: Servers • Network • Storage • Security — والـ Network نفسها جواها Router و Switch.

• S.W فيه: OS • Services • Data.

لما تفكر في أي system، الطبقات دي هي اللي بتحدّد كل حاجة محتاج تشغلها أو تأمنها.

الـ **Data Center** هو المكان اللي بيـ **host** الـ **Infrastructure** كله. لما يبقى عدة مباني بنسقيـه **Zone**، ووقت بنائه هو الـ **Scale**.



KEYNOTE · شرح مفصل

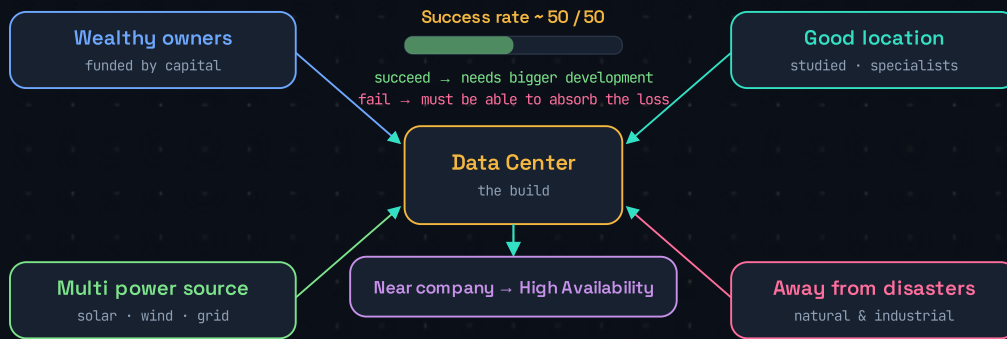
المنظومة اللي بتدير الـ **Infrastructure** اسمها **Data Center**، وهي المكان اللي بيـ **host** البنية التحتية كلها. ممكن تكون تخزين في مكان صغير، ممكن مبنى واحد، وممكن عدة مباني — وأول ما تبقى عدة مباني بنسقيها **Zone**.

الـ **Scale** = وقت البناء:

- أقل حاجة **6 شهور** على الأقل.
- لحد **سنة** يبقى **Data Center** متوسط.
- من **سنتين لـ 3 سنين** يبقى كبير.

أي **scale** بيتطلب تشغيل تدريبي (**operational training**) أقل حاجة سنة. وأقل تكلفة ممكن تبقى **mini center** حتى لو في البيت.

بناء **Data Center** مش شغل فرد ونسبة نجاحه **50/50** — محتاج تمويل، مكان مدروس، أكثر من مصدر طاقة، وبُعد عن الكوارث + قرب من الشركة.



KEYNOTE · شرح مفصل

بناء **Data Center** محتاج متخصصين في كل المجالات، مش شخص واحد. ونسبة نجاح المشروع بتاعت **50/50**:

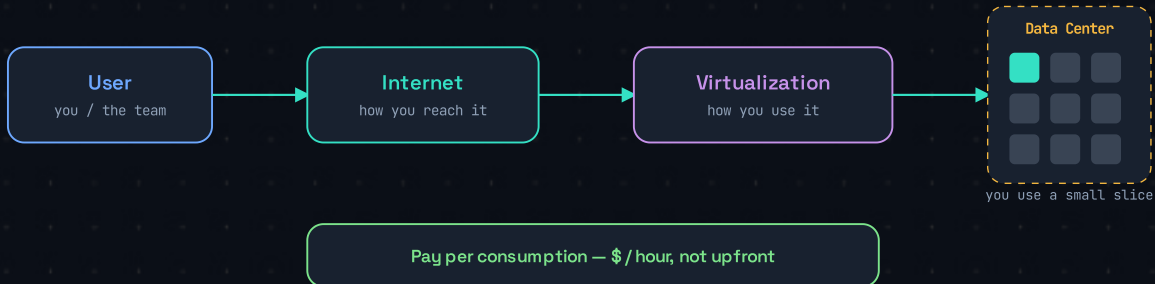
لو نجح: هيتحتاج تطوير أكبر وأطوار جديدة.

لو فشل: لازم يكون عندك القدرة تتحمل الخسائر وترجع ثاني — ولو ما عندكش القدرة يبقى الحل إنك تخرج (OUT).

شروط بناء الـ **Data Centers / Infrastructure**:

- مناسب لأصحاب رؤوس الأموال — بيتعمل بالفلوس.
- مكان كويس ومناسب، مدروس، وليه متخصصين.
- أكثر من مصدر طاقة: **wind · solar** · كورباء عمومية.
- بعيد عن الكوارث الطبيعية والصناعية.
- قريب من الشركة → بيوقر فلوس ويدي **High Availability**.

بتوصل الـ **Data Center** عن طريق الـ **Internet**، وتستخدمه عن طريق الـ **Virtualization** — بتأخذ جزء بسيط وتدفّع على قد استهلاكك.



KEYNOTE · شرح مفصل

بعد ما تجهّز الـ **Data Centers**، بتوصلها إزاي وتستخدمها إزاي؟

- الوصول بيتّم من خلال الـ **Internet**.

- الاستخدام بيتّم من خلال الـ **Virtualization**.

الأكيد إنك مش هتستخدم الـ **Data Center** كله — محتاج جزء بسيط منه، وده بالظبط دور الـ **Virtualization**. وبالتالي مش بتدفع غير على اللي بتستخدمه فعلاً (**pay per hour**).

لو استولكت أقل أو اشترت زيادة، فعلياً انت مش بتخسر — كل اللي بتدفعه على قد استهلاكك من الـ **Resources**.

أي **System** يتكوّن من طبقات، والفرق بين **IaaS** و **PaaS** و **SaaS** هو **مين بيدير كل طبقة** — انت ولا ال **Provider**.

	IaaS	PaaS	SaaS
APP	you	you	provider
Data	you	you	provider
Runtime	you	provider	provider
Middleware	you	provider	provider
Operating SYS	you	provider	provider
Virtualization	provider	provider	provider
Hardware	provider	provider	provider
Storage	provider	provider	provider
Network	provider	provider	provider

☒ You manage
 ☐ Provider manages

KEYNOTE · شرح مفصل

أي **Server** أو **System** يتكوّن من طبقات من فوق لتحت: **APP → Data → Runtime → MiddleWare** → **OS → Virtualization → Hardware → Storage → Network**. الطبقات دي بتتقسم بين **Customer Manage** (انت) و **Provider Manage** (المزوّد)، والحد الفاصل هو الي بيحدّد نوع الخدمة:

- **IaaS (Infrastructure as a Service)**: المزوّد بيديك **Resources** بس، وانت بتشغّل ال **system** بنفسك.

- **PaaS (Platform as a Service)**: بيديك **system** جاهز شغّال (**running**)، وانت بس بتنزل عليه ال **project** بتاعك.

- **SaaS (Software as a Service)**: بيديك **service** جاهزة خالص انت بس بتستخدمها زي **Zoom meeting** — وبتدفع مقابل الاستخدام.

كل ما تنزل لتحت في القائمة (من **IaaS** لـ **SaaS**)، مسؤوليتك بتقل والمزوّد بيدير أكثر.

فيه أكثر من **Provider** كلهم بيّفهموا نفس الحاجة، بس كل واحد مميّز في **service معينة** سواء في الـ **cost** أو غيرّه.

Cloud Providers

same idea · each strongest in a certain service

Amazon
AWS

Microsoft
Azure

Google
GCP

Oracle
OCI

IBM
Cloud

Alibaba
Cloud

KEYNOTE · شرح مفصّل

كل **Provider** بيقدّم **services** وكلهم بيّفهموا نفس الحاجة، لكن كل واحد فيهم مميّز في **service معينة** — سواء من ناحية الـ **cost** أو الأداء أو غيرّه:

- AWS → Amazon
- Azure → Microsoft
- GCP → Google
- OCI → Oracle
- IBM Cloud → IBM
- Alibaba Cloud → Alibaba

خمس أنواع: **Public · Private · Hybrid · Multi-Cloud · Community** — والفرق الجوهرى بين **Private** و **Public** هو الملكية.

1

Public

fully rented from provider

2

Private

= On-Prem · you own it

3

Hybrid

part on-prem + part cloud

4

Multi-Cloud

>1 provider · no single point

5

Community

shared by a group

Private vs Public

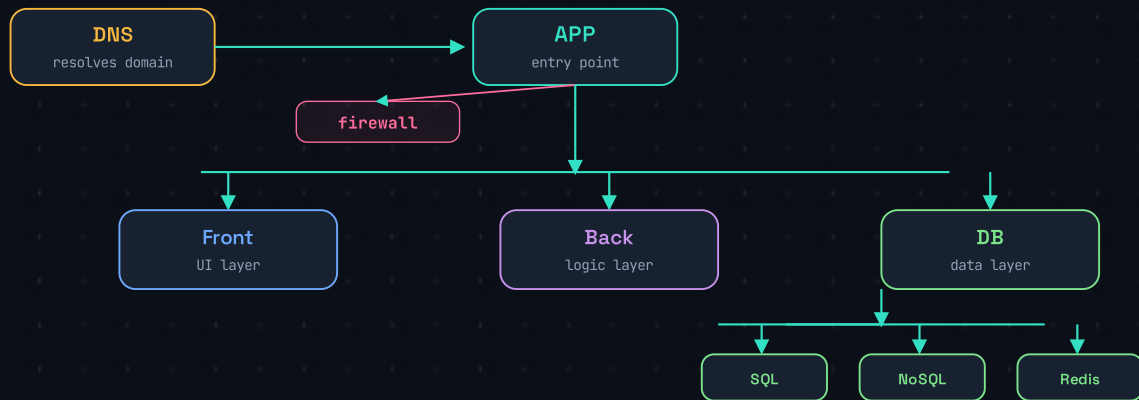
Private = you OWN + manage
Public = you only RENT

KEYNOTE · شرح مفصل

- **Public**: بتأجر كل حاجة من الـ **provider**.
- **Private Cloud**: ده الـ **On-Prem** — انت اللي بتملكه.
- **Hybrid**: جزء **on-prem** وجزء **cloud**.
- **Multi-Cloud**: بتستخدم أكثر من **provider** — لو واحد فيهم وقع في أي وقت، تضمن إن الخدمات ماتوقفش كلها.
- **Community Cloud**: مشتركة بين مجموعة/جهات ليها نفس الاهتمام.

الفرق بين Private و Public: الـ **Private** هو اللي بتملكه انت بالكامل وبتديره وتتحكم فيه، أما الـ **Public** انت بتأجر بس والملكية مش بتاعتك.

عشان تعمل **software** قوي محتاج تعرف الـ **architecture**: **DNS** · **APP** · **Front** / **Back** · **DB** · **firewall**.



KEYNOTE · شرح مفصل

لو عايز **software** قوي، الـ **architecture** بيتكوّن من:

- **DNS** بيحوّل اسم الدومين لعنوان، والـ **APP** هو نقطة الدخول ومحامي بـ **firewall**.
- الـ **APP** بيتقسم: **Front** (الواجهة) · **Back** (المنطق) · **DB** (البيانات).
- الـ **DB** فيه أنواع: **SQL** · **NoSQL** · **Cache (Redis)**.

لو رحت لأي **Provider** هتلاقه بيقدم الـ **services** دي جاهزة (الـ 3 أنواع **DB** جاهزين). عشان كده لازم تكون عارف الـ **architecture** — عشان تعرف تختار أفضل خدمة لكل جزء.

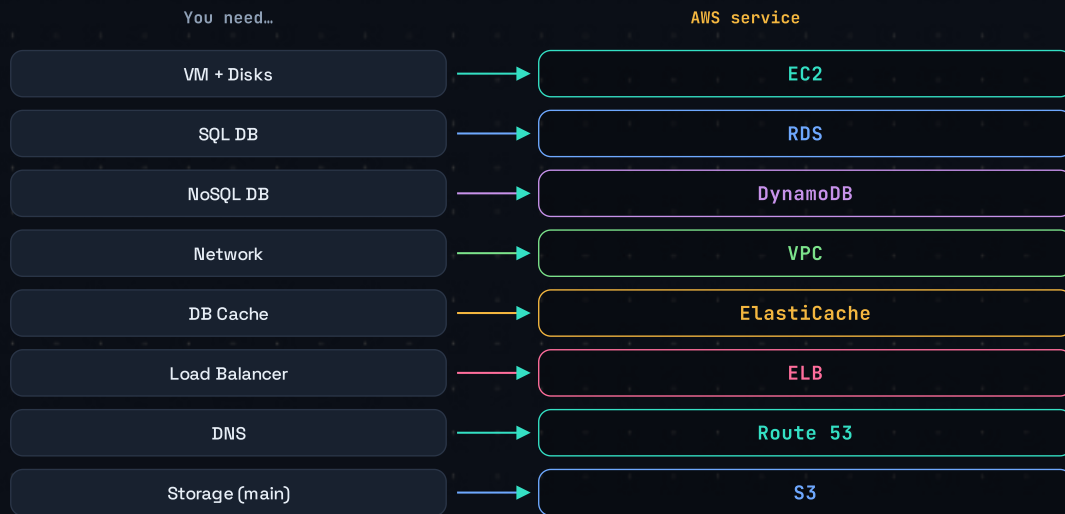
بنية AWS: AZ (أصغر وحدة) → Region (مدينة فيها 3-6 zones) → Edge Location (للمحتوى القريب من المستخدم).



KEYNOTE · شرح مفصل

- **AZ (Availability Zone)**: أصغر وحدة في البنية.
 - **Region**: مدينة محددة فيها مجموعة **Zones**, ويتراوح عددها من 3 إلى 6.
 - **Edge Location**: مسؤولة عن الـ **Content** و **Caching** و **Media**.
- الـ **Media** يحتاج **traffic** عالي جداً مينفعش ينخفض، فبدل ما المستخدم اللي في مصر يجيب المحتوى من **edge location** في أمريكا، يجيبه من أقرب **edge location** ليه. الفكرة كلها إنك تقرب الحاجة للمستخدم قدر الإمكان.

لكل حاجة كنت بتعملها On-Prem فيه service جاهزة في AWS — الجدول ده هو خريطة «محتاج إيه ← أستخدم إيه».

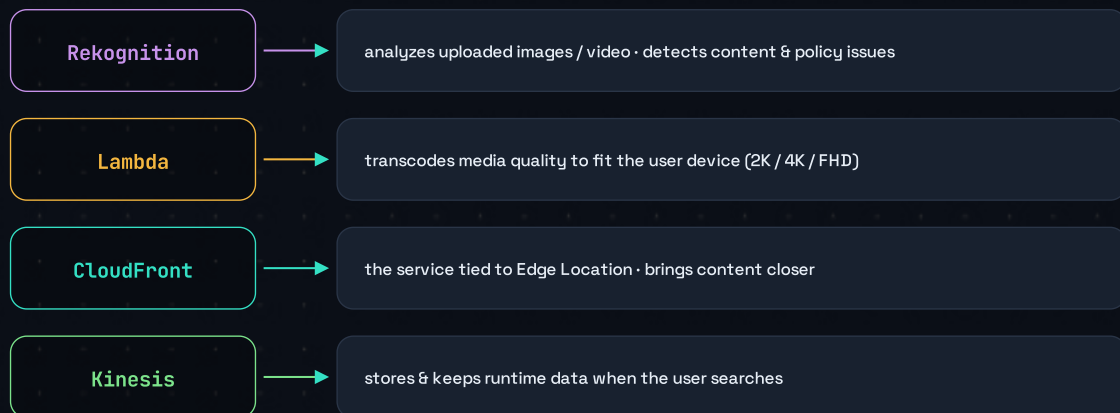


الفكرة إن الـ **APP** يحتاج **Disks + VM** زي الـ **On-Prem**، وأمازون قرّرت تقدّم **service** بتعمل ده بنفسها. فبقى لكل احتياج **service** مقابلة:

- محتاج **EC2 → VM + Disks**
- محتاج **RDS → SQL DB**
- محتاج **DynamoDB → NoSQL DB**
- محتاج **VPC → Network**
- محتاج **ElastiCache → DB Cache**
- محتاج **ELB → Load Balancer**
- محتاج **Route 53 → DNS**
- محتاج **S3 → Storage** أساسي

تصحيح مهم: في الملف الأصلي كان مكتوب **SQL → DynamoDB** و **NoSQL → RDS**، وده مقلوب. الصح — والي في الرسمة هنا — إن **RDS** للـ **SQL / Relational** و **DynamoDB** للـ **NoSQL**.

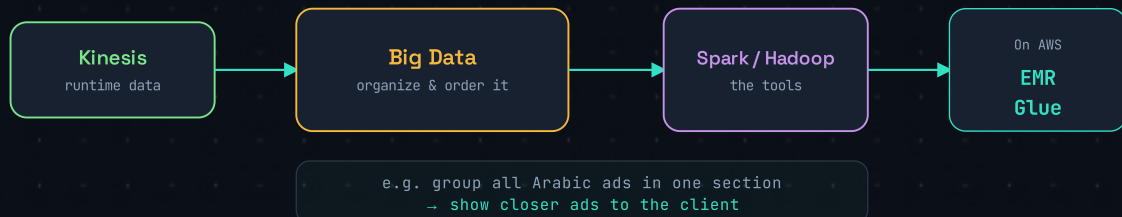
خدمات ذكية للوسائط: **Rekognition** (تحليل) • **Lambda** (تحويل الجودة) • **CloudFront** (Edge) • **Kinesis** (runtime data).



KEYNOTE • شرح مفصل

- **Rekognition**: يبتّل الملفات (صور/فيديو) التي الـ **user** يرفعها — ييكشف مستوى المحتوى وأي مخالفة لسياسة الاستخدام (عنف، محتوى مسيء... إلخ).
- **Lambda**: يبدّل جودة الصورة/الفيديو عشان تتناسب مع الـ **device** بتاع الـ **user** — الموبايل مش هيتعرض له نفس جودة شاشة الـ **2K** أو **4K** أو **FHD**.
- **CloudFront**: هي الـ **service** المرتبطة بالـ **Edge Location**، بتقرب المحتوى للمستخدم (مثلاً مستخدم في أمريكا يتخدم من الـ **edge** بتاعته).
- **Kinesis**: بيخزن ويحفظ الـ **runtime data** لما الـ **user** بيدور على حاجة معينة أو يفتح لها صفحة.

لما ال **Data** بتاعة **Kinesis** تكثر، ال **Big Data** بينظمها ويرتبها — ويدير الحجم الضخم بـ **Spark/Hadoop**، وفي **AWS**: **EMR** و **Glue**.



KEYNOTE • شرح مفصل

لما ال **Data** الخاصة بالـ **Kinesis service** تكثر، فين بتتخزن؟ هنا بييجي دور ال **Big Data** اللي بينظمها ويرتبها.

- بينظمها زي ما يجمع كل الإعلانات العربية في **section** لوحده — الهدف يوريك إعلانات أقرب ليك كـ **client**.
- مع ال **Data** الضخمة محتاج **tools** للإدارة والتنظيم زي **Spark / Hadoop**.
- بالنسبة للـ **Big Data** في **AWS**، ال **tools** بتاعتها هي **EMR** و **Glue**.

نفس الخدمة باسم مختلف عند كل **provider** — مقارنة سريعة بين **AWS** و **Azure** و **GCP**.

AWS		Azure		GCP
EC2	≈	Virtual Machine	≈	Compute Engine
RDS	≈	SQL DB	≈	Cloud Database
Kinesis	≈	Stream Analysis	≈	Cloud Dataflow

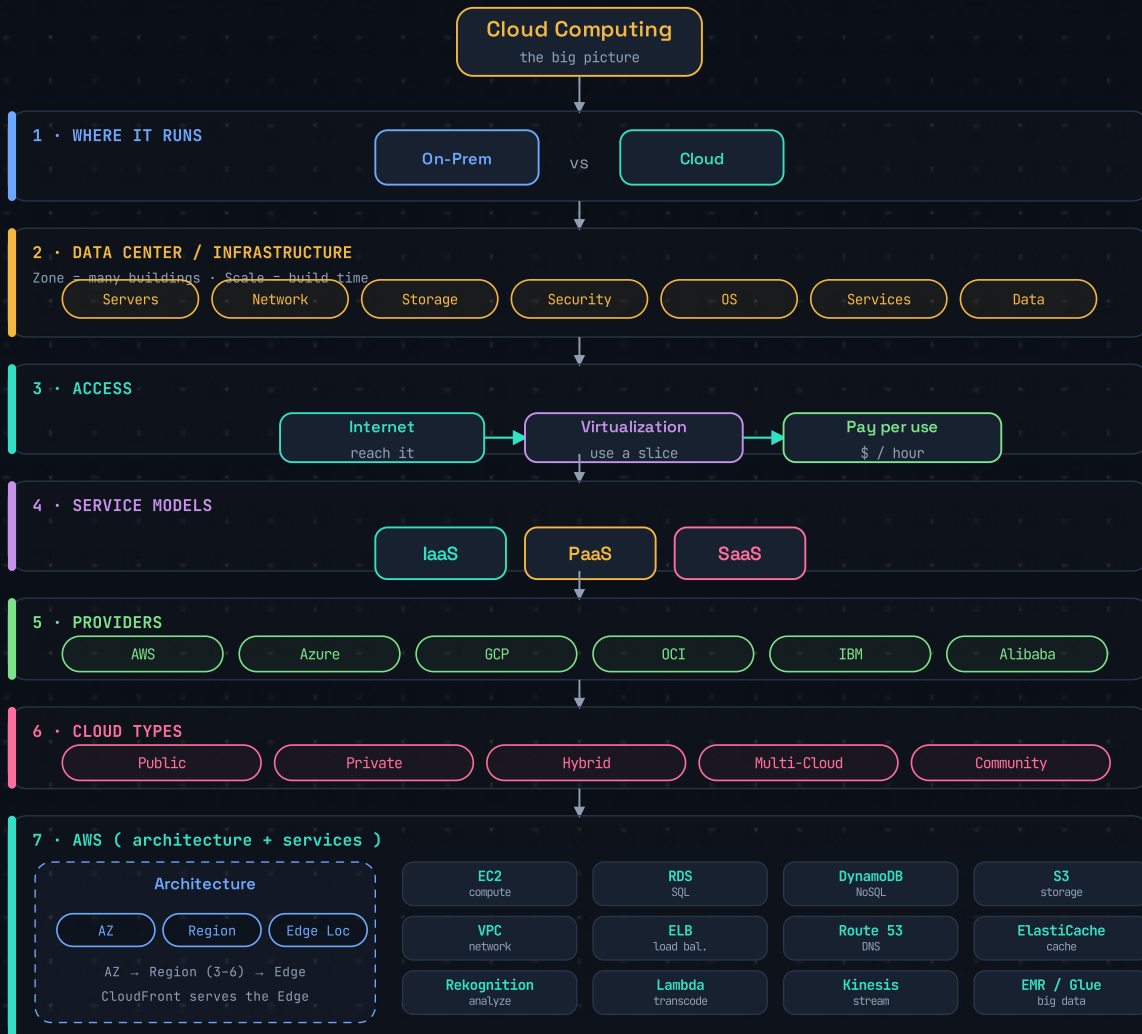
KEYNOTE • شرح مفصل

كل **provider** ييسقي الخدمة باسم مختلف، بس المعنى واحد:

- $\text{Compute Engine} \approx \text{Virtual Machine} \approx \text{EC2} : \text{Compute}$
- $\text{Cloud Database} \approx \text{SQL DB} \approx \text{RDS} : \text{Database}$
- $\text{Cloud Dataflow} \approx \text{Stream Analysis} \approx \text{Kinesis} : \text{Streaming}$

لما تفهم الخدمة عند **provider** واحد، بتقدر تلاقي مقابله عند الباقيين بسهولة — الفروقات في الأسماء مش في الفكرة.

الرسم الشاملة: من الـ **Cloud** كمفهوم، لحد خدمات **AWS** بالتفصيل — كل المواضيع اللي فاتت في خريطة واحدة تربط فهمك مع بعضه.



دي الخلاصة الكاملة في **diagram** واحدة، بتمشي بترتيب الرحلة من فوق لتحت:

- 1 — مكان التشغيل: **On-Prem** ولا **Cloud**.
- 2 — البنية: الـ **Data Center** اللي بيـ **host** الـ **Infrastructure** (Servers/Network/Storage/Security/OS/Services/Data) — وعدة مباني = **Zone**.
- 3 — الوصول: **Internet** للوصول، **Virtualization** للاستخدام، والدفع بالاستهلاك.
- 4 — الموديلز: **IaaS / PaaS / SaaS** حسب مين بيدير الطبقات.
- 5 — المزودين: **AWS • Azure • GCP • OCI • IBM • Alibaba**.
- 6 — الأنواع: **Public • Private • Hybrid • Multi-Cloud • Community**.
- 7 — **AWS**: البنية (AZ → Region → Edge) + الخدمات (EC2, RDS, DynamoDB, S3, VPC, ELB, Route 53, ElastiCache, Rekognition, Lambda, Kinesis, EMR/Glue).

لو قرّبت الشرح، بضّيت على كل **diagram** وشرحه المفصل، وبعدين رجعت للرسمه دي — بتبقى ربطت الصورة كاملة ووصلت لمرحلة إن المعلومة واضحة 100%.

ALnagib

Yousef Salem • DevOps Engineer

CLOUD & AWS • VISUAL NOTES